



Strategy System Documentation

Scope: Complete documentation of the energy management strategy system, including strategy profiles, energy splitting methods, trajectory simulation, and performance analysis for climb trajectory optimization.

Table of Contents

1. [System Overview and Objectives](#)
 2. [Mathematical Foundation](#)
 3. [System Architecture and Data Structures](#)
 4. [Strategy Profiles and Energy Allocation](#)
 5. [Strategy Simulation Engine](#)
 6. [Performance Analysis and Comparison](#)
 7. [Code Execution Flow and Logic](#)
 8. [Integration and Interface](#)
 9. [Validation and Quality Assurance](#)
-

1) System Overview and Objectives

The strategy system implements various energy management approaches for climb trajectories, providing alternative optimization methods to compare against 3D Dynamic Programming results. Each strategy defines how to split available energy between climb rate and acceleration, enabling comprehensive performance analysis across different operational philosophies.

Key Components:

- Energy management strategy profiles
- Strategy simulation and integration
- Performance comparison and analysis

- Data processing and resampling
 - Integration with engine and aerodynamic systems
-

2) Mathematical Foundation

2.1 Energy Management Theory

Theory: The strategy system partitions the commanded specific energy rate between climb energy and speed energy, following the energy approach to aircraft performance.

Mathematical Formulation:

$$E_{total} = E_{climb} + E_{speed} = \text{constant}$$

Where the total energy rate is:

$$\dot{E}_{total} = \dot{E}_{cmd} = 14 \text{ m/s}$$

Energy Split Implementation:

$$\frac{dh}{dt} = w_c \cdot \dot{E}_{cmd} \quad (1)$$

$$\frac{dV}{dt} = \frac{g_0}{V} \cdot w_s \cdot \dot{E}_{cmd} \quad (2)$$

With normalization constraint:

$$w_c + w_s = 1.0$$

Physical Interpretation:

- **Climb weight (w_c):** Fraction of energy dedicated to altitude gain
- **Speed weight (w_s):** Fraction of energy dedicated to acceleration
- **Energy conservation:** Total energy rate remains constant
- **Strategy flexibility:** Different energy allocation philosophies

2.2 Strategy Classification

Strategy Categories:

1. **Linear Strategies:** Fixed energy allocation throughout climb
2. **Exponential Strategies:** Dynamic allocation with exponential curves
3. **Constant Rate Strategies:** Specialized for maintaining specific flight parameters

Strategy Parameters:

- **Altitude Fraction (AF):** Controls strategy behavior
 - **Current Progress:** Altitude progress through climb
 - **Strategy Type:** Determines energy allocation method
-

3) Code Implementation

3.1 StrategyProfiles Class Structure

Class Definition:

```

class StrategyProfiles:
    """
    Energy management strategy profiles for climb trajectories.

    Provides various energy splitting methods for different
    operational philosophies and performance objectives.
    """

    @staticmethod
    def linear_climb(altitude_fraction: float, af: float) -> tuple[float, float]:
        """Linear climb strategy implementation."""

    @staticmethod
    def exp_climb(altitude_fraction: float, af: float) -> tuple[float, float]:
        """Exponential climb strategy implementation."""

    @staticmethod
    def const_speed(altitude_fraction: float, af: float = None) -> tuple[float, float]:
        """Constant speed strategy implementation."""

```

3.2 Linear Strategy Implementation

Theory: Linear strategies maintain constant energy split throughout the climb based on the altitude fraction parameter.

Mathematical Formulation:

$$w_c = AF, \quad w_s = 1 - AF$$

Implementation:

```

@staticmethod
def linear_climb(altitude_fraction: float, af: float) -> tuple[float, float]:
    """
    Linear climb strategy: constant energy split throughout climb.

    Args:
        altitude_fraction: Current altitude progress (unused for linear)
        af: Altitude fraction parameter (0.0 to 1.0)

    Returns:
        tuple: (climb_weight, speed_weight)
    """
    # AF parameter directly specifies climb weight
    cw = float(np.clip(af, 0.0, 1.0))
    sw = 1.0 - cw

    return cw, sw

```

Strategy Variants:

- **AF = 0.10:** Aggressive speed strategy (10% climb, 90% speed)
- **AF = 0.30:** Moderate speed strategy (30% climb, 70% speed)
- **AF = 0.50:** Balanced strategy (50% climb, 50% speed)
- **AF = 0.70:** Moderate climb strategy (70% climb, 30% speed)
- **AF = 0.90:** Aggressive climb strategy (90% climb, 10% speed)

3.3 Exponential Strategy Implementation

Theory: Exponential strategies provide dynamic energy splitting that evolves during the climb using exponential functions.

Mathematical Formulation:

$$\text{exponent} = 1.0 + 2.0 \times \text{AF_param}$$

$$\text{exp_factor} = (\text{current_af})^{\text{exponent}}$$

$$w_c = 0.1 + 0.8 \times \text{exp_factor}$$

$$w_s = 1.0 - w_c$$

Implementation:

```

@staticmethod
def exp_climb(altitude_fraction: float, af: float) -> tuple[float, float]:
    """
    Exponential climb strategy: dynamic energy split with exponential curve.

    Args:
        altitude_fraction: Current altitude progress (0.0 to 1.0)
        af: Altitude fraction parameter controlling curve steepness

    Returns:
        tuple: (climb_weight, speed_weight)
    """
    # Handle parameter input format
    if isinstance(altitude_fraction, tuple):
        af_param, current_af = altitude_fraction
        af_param = float(np.clip(af_param, 0.0, 1.0))
        current_af = float(np.clip(current_af, 0.0, 1.0))
    else:
        af_param = 0.5 # Default parameter
        current_af = float(np.clip(altitude_fraction, 0.0, 1.0))

    # Exponential curve parameters
    min_climb = 0.1 # 10% minimum climb allocation
    max_climb = 0.9 # 90% maximum climb allocation

    # AF parameter controls exponential steepness
    exponent = 1.0 + 2.0 * af_param # Range: 1.0 to 3.0

    # Compute exponential factor
    exp_factor = current_af ** exponent

    # Map to 10%-90% range
    climb_norm = min_climb + (max_climb - min_climb) * exp_factor
    speed_norm = 1.0 - climb_norm

    return climb_norm, speed_norm

```

Strategy Variants:

- **Exp ↑climb AF=0.10**: Gentle transition from speed to climb
- **Exp ↑climb AF=0.30**: Moderate transition from speed to climb
- **Exp ↑climb AF=0.70**: Steep transition from speed to climb
- **Exp ↑speed AF=0.30**: Moderate transition from climb to speed
- **Exp ↑speed AF=0.70**: Steep transition from climb to speed

3.4 Constant Rate Strategies

Constant Speed Strategy:

```
@staticmethod
def const_speed(altitude_fraction: float, af: float = None) -> tuple[float, float]:
    """
    Constant speed strategy: maintain constant true airspeed.

    Args:
        altitude_fraction: Current altitude progress (unused)
        af: Altitude fraction parameter (unused)

    Returns:
        tuple: (1.0, 0.0) - all energy to climb
    """
    # All energy goes to climb; true airspeed maintained by atmospheric effects
    return 1.0, 0.0
```

Constant Mach Strategy:


```
@staticmethod
def const_mach(altitude_fraction: float, af: float = None) -> tuple[float, float]:
    """
    Constant Mach strategy: maintain constant Mach number.

    Args:
        altitude_fraction: Current altitude progress (unused)
        af: Altitude fraction parameter (unused)

    Returns:
        tuple: (1.0, 0.0) - all energy to climb with special integration
    """
    # All energy goes to climb; Mach maintenance through velocity adjustment
    return 1.0, 0.0
```

4) Strategy Simulation System

4.1 StrategyRun Data Structure

Data Container:

```

@dataclass
class StrategyRun:
    """Container for strategy simulation results."""
    label: str # Strategy name
    alt_m: np.ndarray # Altitude history (m)
    mach: np.ndarray # Mach number history
    time_s: np.ndarray # Time history (s)
    lever: np.ndarray # Lever history (0-1)
    T_total_N: np.ndarray # Total thrust (N)
    D_N: np.ndarray # Drag (N)
    Ps_mps: np.ndarray # Specific excess power (m/s)
    mdot_kgps: np.ndarray # Fuel flow rate (kg/s)
    dt_s: np.ndarray # Time steps (s)
    dFuel_kg: np.ndarray # Fuel consumed per step (kg)
    cumFuel_kg: np.ndarray # Cumulative fuel (kg)
    thrust_limited: np.ndarray # Thrust limitation flags
    fuel_total_kg: float # Total fuel consumed (kg)

```

4.2 Strategy Simulation Implementation

Core Simulation Function:

```

def simulate_strategy_path(*, label: str, aero: AeroTables, eng: EngineWrapper,
                           mass0_kg: float, h0_m: float, V0_ms: float,
                           target_alt_m: float, dt: float,
                           strategy_fn: Callable, altitude_fraction: Optional[float] = None) -> StrategyRun:
    """
    Simulate a climbing strategy path from initial conditions to target altitude.

    This is the core strategy simulation function that integrates the climb trajectory
    using the specified strategy function.

    Args:
        label: Strategy label for identification
        aero: Aerodynamics tables
        eng: Engine wrapper
        mass0_kg: Initial mass [kg]
        h0_m: Initial altitude [m]
        V0_ms: Initial velocity [m/s]
        target_alt_m: Target altitude [m]
        dt: Time step [s]
        strategy_fn: Strategy function that returns (cw, sw) weights
        altitude_fraction: Altitude fraction parameter for strategy

    Returns:
        StrategyRun: Complete simulation results
    """

```

4.2.1 Strategy Simulation Flow

Complete Logical Flow:

STRATEGY SIMULATION FLOW

START: `simulate_strategy_path(label, aero, eng, mass0_kg, h0_m, V0_ms, target_alt_m, dt, strategy_fn)`

↓

Step 1: Initialize State

- └ Initialize state variables: `h_hist, V_hist, time_hist`
- └ Initialize result arrays: `lever_hist, T_total_hist, D_hist`
- └ Initialize fuel tracking: `dFuel_hist, cumFuel_hist`
- └ Set initial conditions: `h_hist[0] = h0_m, V_hist[0] = V0_ms`
- └ Calculate aircraft weight: $W = \text{mass0_kg} \times g0$

↓

Step 2: Main Simulation Loop

WHILE `h_hist[-1] < target_alt_m`:

- └ Get current state: `h = h_hist[-1], V = V_hist[-1]`
- └ Calculate altitude progress: $\text{current_altitude_fraction} = (h - h0_m) / (target_alt_m - h0_m)$
- └ Prepare strategy parameters:
 - └ IF "Linear" in label: `strategy_altitude_fraction = altitude_fraction`
 - └ ELIF "Exp" in label: `strategy_altitude_fraction = (altitude_fraction, current_altitude_fraction)`
 - └ ELSE: `strategy_altitude_fraction = None`
- └ Get energy split: `cw, sw = strategy_fn(h, V, strategy_altitude_fraction)`
- └ Normalize weights: $w_c, w_s = cw / (cw + sw), sw / (cw + sw)$
- └ Apply energy split to kinematics:
 - └ IF constant Mach strategy:
 - └ Get atmospheric properties: `T, P, rho = isa_properties(h)`
 - └ Calculate speed of sound: $a = a_from_T(T)$
 - └ Calculate current Mach: $\text{current_mach} = V / a$
 - └ Calculate speed of sound gradient: $\text{dadh} = (a2 - a1) / \text{eps}$
 - └ Climb rate: $\text{dh_dt} = w_c \times E_DOT_CMD$
 - └ Velocity rate: $\text{dv_dt} = \text{current_mach} \times \text{dadh} \times \text{dh_dt}$
 - └ ELSE (standard energy split):

```

| |   └─ Climb rate:  $dh\_dt = w\_c \times E\_DOT\_CMD$  |
| |   └─ Velocity rate:  $dv\_dt = (w\_s \times E\_DOT\_CMD \times g0) / V$  |
| |   |
| |   └─ Calculate required thrust: |
| |   └─ Energy rate:  $E\_DOT = dh\_dt + (V / g0) \times dv\_dt$  |
| |   └─ Get drag:  $D = aero.get\_drag(V / a, h)$  |
| |   └─ Required thrust:  $F\_required\_total = D + (E\_DOT \times W) / V$  |
| |   |
| |   └─ Solve for lever position: |
| |   └─ lever, T_per, limited = find_lever_for_thrust(eng, F_required_total, V/a, h) |
| |   └─ Align engine state: eng.thrust_with_lever(lever, V/a, h) |
| |   └─ Get TSFC: tsfc = eng.tsfc_current() |
| |   |
| |   └─ Calculate fuel flow: |
| |   └─ IF tsfc > 0:  $mdot\_total = tsfc \times T\_per \times N\_ENGINES$  |
| |   └─ ELSE:  $mdot\_total = 0.0$  |
| |   |
| |   └─ Calculate specific excess power: |
| |   └─  $T\_total = T\_per \times N\_ENGINES$  |
| |   └─  $Ps = ((T\_total - D) \times V) / W$  |
| |   |
| |   └─ Calculate time step: |
| |   └─ IF  $dh\_dt > 0$ :  $dt\_actual = \min(dt, (target\_alt\_m - h) / dh\_dt)$  |
| |   └─ ELSE:  $dt\_actual = dt$  |
| |   |
| |   └─ Update states: |
| |   └─  $h\_new = h + dh\_dt \times dt\_actual$  |
| |   └─  $V\_new = V + dv\_dt \times dt\_actual$  |
| |   └─  $time\_new = time\_hist[-1] + dt\_actual$  |
| |   |
| |   └─ Calculate fuel consumption: |
| |   └─  $dFuel = mdot\_total \times dt\_actual$  |
| |   └─  $current\_fuel += dFuel$  |
| |   |
| |   └─ Store results: |
| |   └─ Append to all history arrays |
| |   └─ CONTINUE loop |
| |   └─

```

↓

```

└─

```

```
|   Step 3: Create StrategyRun   |
|   └─ Calculate final Mach numbers: mach = [V / a_from_T(isa_properties(h)[0]) for V, h in zip(V_h, h)]
|   └─ Create StrategyRun object with all results
|   └─ RETURN: StrategyRun
└────────────────────────────────┘
```

↓

END: Return complete simulation results

Simulation Loop:

```

def simulate_strategy_path(*, label: str, aero: AeroTables, eng: EngineWrapper,
                           mass0_kg: float, h0_m: float, V0_ms: float,
                           target_alt_m: float, dt: float,
                           strategy_fn: Callable, altitude_fraction: Optional[float] = None) -> Strategy:
    """Strategy simulation implementation."""

    # Initialize state variables
    h_hist = [h0_m]
    V_hist = [V0_ms]
    time_hist = [0.0]
    lever_hist = []
    T_total_hist = []
    D_hist = []
    Ps_hist = []
    mdot_hist = []
    dt_hist = []
    dFuel_hist = []
    cumFuel_hist = [0.0]
    thrust_limited_hist = []

    # Simulation parameters
    W = mass0_kg * g0 # Aircraft weight
    current_fuel = 0.0

    # Main simulation loop
    while h_hist[-1] < target_alt_m:
        h = float(h_hist[-1])
        V = float(V_hist[-1])

        # Calculate current altitude progress
        current_altitude_fraction = (h - h0_m) / (target_alt_m - h0_m) if target_alt_m > h0_m else 0.0
        current_altitude_fraction = np.clip(current_altitude_fraction, 0.0, 1.0)

        # Prepare strategy input parameters
        if "Linear" in label:
            strategy_altitude_fraction = altitude_fraction # Use AF parameter directly
        elif "Exp" in label:
            strategy_altitude_fraction = (altitude_fraction, current_altitude_fraction) # (AF_parameter, current_altitude_fraction)
        else: # Constant strategies

```

```

strategy_altitude_fraction = None

# Get energy split weights
cw, sw = strategy_fn(h, V, strategy_altitude_fraction)

# Normalize weights
s = max(cw + sw, 1e-12)
w_c, w_s = cw / s, sw / s

# Apply energy split to kinematics
if getattr(strategy_fn, "_const_mach", False):
    # Constant Mach: special velocity integration
    T_air, _, _ = isa_properties(h)
    a = a_from_T(T_air)
    current_mach = V / max(a, 1e-9)

    # Calculate speed of sound gradient
    eps = max(1.0, h * 0.001)
    a1 = a_from_T(isa_properties(h - eps/2)[0])
    a2 = a_from_T(isa_properties(h + eps/2)[0])
    dadh = (a2 - a1) / eps

    dh_dt = w_c * E_DOT_CMD
    dv_dt = current_mach * dadh * dh_dt
else:
    # Standard energy split
    dh_dt = w_c * E_DOT_CMD
    dv_dt = (w_s * E_DOT_CMD * g0) / max(V, 1e-9)

# Calculate required thrust
E_DOT = dh_dt + (V / g0) * dv_dt
D = aero.get_drag(V / max(a_from_T(isa_properties(h)[0]), 1e-9), h)
F_required_total = D + (E_DOT * W) / max(V, 1e-9)

# Solve for lever position
lever, T_per, limited = find_lever_for_thrust(eng, F_required_total, V / max(a_from_T(isa_

# Align engine state and get TSFC
eng.thrust_with_lever(lever, V / max(a_from_T(isa_properties(h)[0]), 1e-9), h)
tsfc = eng.tsfc_current()

```



```

# Calculate fuel flow
if tsfc > 0:
    mdot_total = tsfc * T_per * N_ENGINES
else:
    mdot_total = 0.0

# Calculate specific excess power
T_total = T_per * N_ENGINES
Ps = ((T_total - D) * V) / W

# Time step calculation
if dh_dt > 0:
    dt_actual = min(dt, (target_alt_m - h) / dh_dt)
else:
    dt_actual = dt

# Update states
h_new = h + dh_dt * dt_actual
V_new = V + dv_dt * dt_actual
time_new = time_hist[-1] + dt_actual

# Calculate fuel consumption
dFuel = mdot_total * dt_actual
current_fuel += dFuel

# Store results
h_hist.append(h_new)
V_hist.append(V_new)
time_hist.append(time_new)
lever_hist.append(lever)
T_total_hist.append(T_total)
D_hist.append(D)
Ps_hist.append(Ps)
mdot_hist.append(mdot_total)
dt_hist.append(dt_actual)
dFuel_hist.append(dFuel)
cumFuel_hist.append(current_fuel)
thrust_limited_hist.append(limited)

```

```
# Create StrategyRun object
return StrategyRun(
    label=label,
    alt_m=np.array(h_hist),
    mach=np.array([V / max(a_from_T(isa_properties(h)[0]), 1e-9) for V, h in zip(V_hist, h_hist)]),
    time_s=np.array(time_hist),
    lever=np.array(lever_hist),
    T_total_N=np.array(T_total_hist),
    D_N=np.array(D_hist),
    Ps_mps=np.array(Ps_hist),
    mdot_kgps=np.array(mdot_hist),
    dt_s=np.array(dt_hist),
    dFuel_kg=np.array(dFuel_hist),
    cumFuel_kg=np.array(cumFuel_hist),
    thrust_limited=np.array(thrust_limited_hist),
    fuel_total_kg=current_fuel
)
```

5) Strategy Building and Configuration

5.1 Strategy Set Generation

Strategy Builder:

```

def build_strategy_set() -> List[tuple[str, Callable, Optional[float]]]:
    """
    Build the complete set of climbing strategies to test.

    Returns a list of (name, function, altitude_fraction) tuples for all
    available climbing strategies.

    Returns:
        List of strategy definitions
    """
    afs = [0.10, 0.30, 0.50, 0.70, 0.90] # AF parameter values
    out = []

    # Linear strategies (5 variants)
    for af in afs:
        out.append((f"Linear AF={af:.2f}", StrategyProfiles.linear_climb, af))

    # Exponential strategies (10 variants: 2 types × 5 AF values)
    for af in afs:
        out.append((f"Exp ↑climb AF={af:.2f}", StrategyProfiles.exp_climb, af))
    for af in afs:
        out.append((f"Exp ↑speed AF={af:.2f}", StrategyProfiles.exp_speed, af))

    # Constant rate strategies (2 variants)
    out.append(("Constant speed", StrategyProfiles.const_speed, None))
    out.append(("Constant Mach", StrategyProfiles.const_mach, None))

    return out

```

Generated Strategy Set:

- **5 Linear strategies:** $AF \in \{0.10, 0.30, 0.50, 0.70, 0.90\}$
- **10 Exponential strategies:** 2 types × 5 AF values
- **2 Constant rate strategies:** Speed and Mach variants
- **Total: 17 strategies**

5.2 Strategy Execution Framework

Strategy Simulation Loop:

```

def run_strategy_comparison(aero: AeroTables, eng: EngineWrapper,
                           mass0_kg: float, h0_m: float, V0_ms: float,
                           target_alt_m: float, dt: float) -> List[StrategyRun]:
    """Run complete strategy comparison."""

    strategies = build_strategy_set()
    results = []

    for name, strategy_fn, altitude_fraction in strategies:
        print(f"Running strategy: {name}")

        try:
            sr = simulate_strategy_path(
                label=name,
                aero=aero,
                eng=eng,
                mass0_kg=mass0_kg,
                h0_m=h0_m,
                V0_ms=V0_ms,
                target_alt_m=target_alt_m,
                dt=dt,
                strategy_fn=strategy_fn,
                altitude_fraction=altitude_fraction
            )

            results.append(sr)
            print(f" Fuel: {sr.fuel_total_kg:.1f} kg, Time: {sr.time_s[-1]/60:.1f} min")

        except Exception as e:
            print(f" Failed: {e}")

    return results

```

6) Data Processing and Resampling

6.1 Strategy Resampling System

Resampling Function:

```

def resample_strategy_run(sr: StrategyRun, n_samples: int) -> StrategyRun:
    """
    Resample a strategy run to a specified number of points.

    Args:
        sr: Original strategy run
        n_samples: Number of samples in resampled run

    Returns:
        Resampled strategy run
    """
    # Generate uniform altitude grid
    n = max(2, n_samples)
    alt_new = np.linspace(sr.alt_m[0], sr.alt_m[-1], n)

    # Define interpolation function
    def interp(y):
        return np.interp(alt_new, sr.alt_m, y)

    # Interpolate continuous variables
    time_new = interp(sr.time_s)
    mach_new = interp(sr.mach)
    lever_new = interp(sr.lever)
    Ttot_new = interp(sr.T_total_N)
    D_new = interp(sr.D_N)
    Ps_new = interp(sr.Ps_mps)
    mdot_new = interp(sr.mdot_kgps)
    cumF_new = interp(sr.cumFuel_kg)

    # Handle boolean variable
    limited_f = np.interp(alt_new, sr.alt_m, sr.thrust_limited.astype(float))
    limited_new = (limited_f >= 0.5)

    # Compute derived variables
    dt_new = np.diff(time_new, prepend=time_new[0])
    dFuel_new = np.diff(cumF_new, prepend=cumF_new[0])
    fuel_tot = float(cumF_new[-1] - cumF_new[0])

    # Create resampled StrategyRun

```

```
return StrategyRun(  
    label=sr.label,  
    alt_m=alt_new,  
    mach=mach_new,  
    time_s=time_new,  
    lever=lever_new,  
    T_total_N=Ttot_new,  
    D_N=D_new,  
    Ps_mps=Ps_new,  
    mdot_kgps=mdot_new,  
    dt_s=dt_new,  
    dFuel_kg=dFuel_new,  
    cumFuel_kg=cumF_new,  
    thrust_limited=limited_new,  
    fuel_total_kg=fuel_tot  
)
```

6.1.1 Strategy Resampling Flow

Complete Logical Flow:

STRATEGY RESAMPLING FLOW

START: `resample_strategy_run(sr: StrategyRun, n_samples: int)`



Step 1: Generate Uniform Grid

└ `n = max(2, n_samples)`

└ `alt_new = np.linspace(sr.alt_m[0], sr.alt_m[-1], n)`



Step 2: Define Interpolation

└ `interp(y) = np.interp(alt_new, sr.alt_m, y)`

└ HELPER: Reusable interpolation function



Step 3: Interpolate Continuous

Step 3: Interpolate Continuous

└ `time_new = interp(sr.time_s)`

└ `mach_new = interp(sr.mach)`

└ `lever_new = interp(sr.lever)`

└ `Ttot_new = interp(sr.T_total_N)`

└ `D_new = interp(sr.D_N)`

└ `Ps_new = interp(sr.Ps_mps)`

└ `mdot_new = interp(sr.mdot_kgps)`

└ `cumF_new = interp(sr.cumFuel_kg)`



Step 4: Handle Boolean Variable

└ `limited_f = np.interp(alt_new, sr.alt_m, sr.thrust_limited.astype(float))`

└ `limited_new = (limited_f >= 0.5)`

└ CONVERT: Boolean → float → interpolate → boolean




```

|   Step 5: Compute Derived Variables |
|   └─ dt_new = np.diff(time_new, prepend=time_new[0]) |
|   └─ dFuel_new = np.diff(cumF_new, prepend=cumF_new[0]) |
|   └─ fuel_tot = float(cumF_new[-1] - cumF_new[0]) |
└──────────────────────────────────────────────────────────┘

↓

|   Step 6: Create Resampled Run   |
|   └─ RETURN: StrategyRun( |
|   |   └─ label=sr.label (unchanged) |
|   |   └─ alt_m=alt_new (uniform grid) |
|   |   └─ mach=mach_new (interpolated) |
|   |   └─ time_s=time_new (interpolated) |
|   |   └─ lever=lever_new (interpolated) |
|   |   └─ T_total_N=Ttot_new (interpolated) |
|   |   └─ D_N=D_new (interpolated) |
|   |   └─ Ps_mps=Ps_new (interpolated) |
|   |   └─ mdot_kgps=mdot_new (interpolated) |
|   |   └─ dt_s=dt_new (derived) |
|   |   └─ dFuel_kg=dFuel_new (derived) |
|   |   └─ cumFuel_kg=cumF_new (interpolated) |
|   |   └─ thrust_limited=limited_new (boolean interpolated) |
|   |   └─ fuel_total_kg=fuel_tot (total fuel consumption) |
|   |   ) |
└──────────────────────────────────────────────────────────┘

↓

END: Return resampled StrategyRun with uniform altitude grid

```

6.2 Performance Analysis

Strategy Comparison:

```

def analyze_strategy_performance(strategies: List[StrategyRun]) -> dict:
    """Analyze and compare strategy performance."""

    # Extract performance metrics
    fuel_consumptions = [s.fuel_total_kg for s in strategies]
    times_min = [s.time_s[-1] / 60.0 for s in strategies]
    final_machs = [s.mach[-1] if len(s.mach) > 0 else 0 for s in strategies]
    avg_ps = [np.mean(s.Ps_mps) for s in strategies]

    # Rank by fuel efficiency
    fuel_ranking = sorted(enumerate(strategies), key=lambda x: x[1].fuel_total_kg)

    # Calculate statistics
    stats = {
        'best_fuel': min(fuel_consumptions),
        'worst_fuel': max(fuel_consumptions),
        'best_time': min(times_min),
        'worst_time': max(times_min),
        'fuel_spread': max(fuel_consumptions) - min(fuel_consumptions),
        'time_spread': max(times_min) - min(times_min),
        'ranking': fuel_ranking
    }

    return stats

```

7) Integration Examples

7.1 Basic Strategy Usage

```
from climb import StrategyProfiles, simulate_strategy_path, build_strategy_set

# Load system components
aero = AeroTables("aerodynamics.xlsx")
eng = EngineWrapper("engine_stubs")

# Run a single strategy
sr = simulate_strategy_path(
    label="Linear AF=0.50",
    aero=aero,
    eng=eng,
    mass0_kg=65000.0,
    h0_m=0.0,
    V0_ms=150.0,
    target_alt_m=10000.0,
    dt=1.0,
    strategy_fn=StrategyProfiles.linear_climb,
    altitude_fraction=0.50
)

print(f"Strategy: {sr.label}")
print(f"Fuel consumed: {sr.fuel_total_kg:.1f} kg")
print(f"Flight time: {sr.time_s[-1]/60:.1f} minutes")
print(f"Final Mach: {sr.mach[-1]:.3f}")
```

7.2 Strategy Comparison

```
# Run complete strategy comparison
strategies = build_strategy_set()
results = []

for name, strategy_fn, af in strategies:
    sr = simulate_strategy_path(
        label=name,
        aero=aero,
        eng=eng,
        mass0_kg=65000.0,
        h0_m=0.0,
        V0_ms=150.0,
        target_alt_m=10000.0,
        dt=1.0,
        strategy_fn=strategy_fn,
        altitude_fraction=af
    )
    results.append(sr)

# Analyze results
stats = analyze_strategy_performance(results)

print("Strategy Performance Ranking:")
for i, (idx, strategy) in enumerate(stats['ranking'][:5]):
    print(f"{i+1}. {strategy.label}: {strategy.fuel_total_kg:.1f} kg")
```

7.3 Strategy Visualization

```

import matplotlib.pyplot as plt

# Plot strategy trajectories
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Select top 5 strategies
top_strategies = [s for _, s in stats['ranking'][:5]]

for sr in top_strategies:
    # Altitude vs Time
    axes[0, 0].plot(sr.time_s/60, sr.alt_m/1000, label=sr.label)

    # Mach vs Altitude
    axes[0, 1].plot(sr.alt_m/1000, sr.mach, label=sr.label)

    # Fuel vs Time
    axes[1, 0].plot(sr.time_s/60, sr.cumFuel_kg, label=sr.label)

    # Lever vs Altitude
    axes[1, 1].plot(sr.alt_m/1000, sr.lever, label=sr.label)

# Format plots
axes[0, 0].set_xlabel('Time (min)')
axes[0, 0].set_ylabel('Altitude (km)')
axes[0, 0].set_title('Altitude vs Time')
axes[0, 0].legend()
axes[0, 0].grid(True)

axes[0, 1].set_xlabel('Altitude (km)')
axes[0, 1].set_ylabel('Mach Number')
axes[0, 1].set_title('Mach vs Altitude')
axes[0, 1].legend()
axes[0, 1].grid(True)

axes[1, 0].set_xlabel('Time (min)')
axes[1, 0].set_ylabel('Cumulative Fuel (kg)')
axes[1, 0].set_title('Fuel Consumption')
axes[1, 0].legend()
axes[1, 0].grid(True)

```

```
axes[1, 1].set_xlabel('Altitude (km)')
axes[1, 1].set_ylabel('Lever Position')
axes[1, 1].set_title('Thrust Lever vs Altitude')
axes[1, 1].legend()
axes[1, 1].grid(True)

plt.tight_layout()
plt.show()
```

8) Performance Characteristics

8.1 Typical Performance Results

Strategy Performance Table (8000m climb, initial mass 65,000 kg):

Strategy Type	Fuel (kg)	Time (min)	Final Mach	Characteristics
Linear AF=0.10	~665	6.2	0.72	Fast acceleration, slow climb
Linear AF=0.30	~655	6.5	0.71	Moderate acceleration/climb balance
Linear AF=0.50	~650	6.8	0.70	Balanced energy distribution
Linear AF=0.70	~652	7.1	0.69	Climb-focused with some acceleration
Linear AF=0.90	~660	7.5	0.68	Aggressive climb, minimal acceleration
Exp ↑climb AF=0.30	~645	6.6	0.70	Natural acceleration-to-climb transition
Exp ↑climb AF=0.70	~648	6.9	0.69	Steep acceleration-to-climb transition

Strategy Type	Fuel (kg)	Time (min)	Final Mach	Characteristics
Exp ↑speed AF=0.30	~652	6.4	0.71	Natural climb-to-acceleration transition
Exp ↑speed AF=0.70	~658	6.3	0.72	Steep climb-to-acceleration transition
Constant speed	~680	7.8	0.61	Steady climb, decreasing Mach
Constant Mach	~675	7.2	0.50	Constant Mach, decreasing speed

8.2 Selection Guidelines

For Fuel Efficiency:

1. **Exp ↑climb AF=0.30:** Best overall fuel efficiency with natural transition
2. **Linear AF=0.50:** Reliable balanced approach
3. **Exp ↑speed AF=0.30:** Good efficiency with early altitude gain

For Time Efficiency:

1. **Exp ↑speed AF=0.70:** Fast acceleration then climb
2. **Linear AF=0.10:** Rapid speed buildup
3. **Exp ↑speed AF=0.30:** Balanced time performance

For Operational Constraints:

1. **Constant Mach:** When specific Mach maintenance is required
 2. **Constant speed:** For missions requiring constant airspeed
 3. **Linear strategies:** When predictable, constant energy split is needed
-

9) Advanced Features

9.1 Custom Strategy Development

Custom Strategy Template:

```
def custom_strategy(altitude: float, velocity: float, altitude_fraction: float) -> tuple[float, float]:
    """
    Custom strategy implementation template.

    Args:
        altitude: Current altitude (m)
        velocity: Current true airspeed (m/s)
        altitude_fraction: Strategy parameter or progress

    Returns:
        tuple: (climb_weight, speed_weight)
    """
    # Implement custom energy allocation logic
    # Example: sinusoidal energy split
    import math

    progress = altitude_fraction if isinstance(altitude_fraction, (int, float)) else 0.5
    progress = np.clip(progress, 0.0, 1.0)

    # Sinusoidal variation
    cw = 0.5 + 0.4 * math.sin(2 * math.pi * progress)
    sw = 1.0 - cw

    return cw, sw

# Add to strategy set
custom_strategies = [
    ("Custom Sinusoidal", custom_strategy, None)
]
```

9.2 Adaptive Strategy System

Adaptive Strategy Framework:

```

class AdaptiveStrategy:
    """Adaptive strategy that adjusts based on flight conditions."""

    def __init__(self, base_strategy, adaptation_rules):
        self.base_strategy = base_strategy
        self.adaptation_rules = adaptation_rules
        self.performance_history = []

    def __call__(self, altitude: float, velocity: float, altitude_fraction: float) -> tuple[float, float]:
        """Adaptive strategy execution."""

        # Get base strategy weights
        cw_base, sw_base = self.base_strategy(altitude, velocity, altitude_fraction)

        # Apply adaptations based on rules
        cw_adapted, sw_adapted = self._apply_adaptations(
            cw_base, sw_base, altitude, velocity, altitude_fraction
        )

        return cw_adapted, sw_adapted

    def _apply_adaptations(self, cw: float, sw: float, altitude: float,
                           velocity: float, altitude_fraction: float) -> tuple[float, float]:
        """Apply adaptation rules to base strategy."""

        # Example: Increase climb weight at high altitude
        if altitude > 8000: # Above 8 km
            cw *= 1.2
            sw = 1.0 - cw

        # Example: Reduce speed weight at low Mach
        mach = velocity / max(a_from_T(isa_properties(altitude)[0]), 1e-9)
        if mach < 0.4:
            sw *= 0.8
            cw = 1.0 - sw

        return cw, sw

```

10) Performance Analysis

10.1 Computational Complexity

Strategy Simulation:

- **Time complexity:** $O(N_{\text{steps}})$ per strategy
- **Per step:** One lever solve (≤ 21 samples) + one TSFC query
- **Memory:** $O(N_{\text{steps}})$ for history arrays

Strategy Comparison:

- **Time complexity:** $O(S \times N_{\text{steps}})$ where S = number of strategies
- **Memory:** $O(S \times N_{\text{steps}})$ for all strategy results
- **Typical performance:** 1-10 seconds for complete comparison

10.2 Benchmarking Results

Typical Performance:

```

# Benchmark strategy simulation
import time

# Test conditions
test_strategies = build_strategy_set()[:5] # First 5 strategies

start_time = time.time()
results = []
for name, strategy_fn, af in test_strategies:
    sr = simulate_strategy_path(
        label=name,
        aero=aero,
        eng=eng,
        mass0_kg=65000.0,
        h0_m=0.0,
        V0_ms=150.0,
        target_alt_m=10000.0,
        dt=1.0,
        strategy_fn=strategy_fn,
        altitude_fraction=af
    )
    results.append(sr)

total_time = time.time() - start_time
print(f"5 strategies simulated in {total_time:.2f} seconds")
print(f"Average time per strategy: {total_time/5:.2f} seconds")

```

Expected Results:

- **Single strategy:** 0.5-2.0 seconds
 - **Complete comparison:** 5-20 seconds
 - **Memory usage:** 10-50 MB for typical results
-

11) Future Enhancements

11.1 Advanced Strategy Types

Potential Improvements:

- **Machine learning strategies:** AI-optimized energy allocation
- **Multi-objective strategies:** Pareto-optimal fuel/time trade-offs
- **Weather-adaptive strategies:** Dynamic adjustment for conditions
- **Mission-specific strategies:** Tailored for specific flight phases

11.2 Real-time Strategy Selection

Potential Improvements:

- **Online optimization:** Real-time strategy adjustment
 - **Performance prediction:** Forecast strategy performance
 - **Adaptive selection:** Automatic best-strategy selection
 - **Multi-aircraft coordination:** Fleet-level strategy optimization
-

12) References and Standards

12.1 Energy Management Standards

- **ICAO Standards:** International energy management guidelines
- **FAA Regulations:** Energy approach flight procedures
- **SAE Aerospace:** Energy management best practices
- **AIAA Standards:** Energy optimization methods

12.2 Technical References

1. **Anderson, J.D.** - "Introduction to Flight" - McGraw-Hill
2. **Raymer, D.P.** - "Aircraft Design: A Conceptual Approach" - AIAA
3. **Torenbeek, E.** - "Synthesis of Subsonic Airplane Design" - Delft University Press

4. **Roskam, J.** - "Airplane Design" - DARcorporation

End of Strategy System Documentation